Créer une application avec SpringBoot

A. Explication du TP

Objectif du TP

Créer une application web simple avec Spring Boot qui permet de gérer un catalogue de produits (CRUD : Créer, Lire, Mettre à jour, Supprimer). Ce premier TP en est la **préparation** : on installe l'environnement, on crée le projet, on le lance et on apprend à le configurer. Le CRUD lui-même (classe *Produit*, contrôleur REST) sera réalisé dans les TP suivants.

Notions et outils à connaître

- **Spring Boot** : framework Java qui simplifie la création d'applications (configuration automatique, serveur intégré).
- **STS (Spring Tools 4)** : IDE basé sur Eclipse, optimisé pour Spring Boot ; il intègre Spring Initializr pour générer un projet.
- **Maven** : outil qui gère les dépendances et la compilation du projet à partir du fichier `pom.xml`.
- **API REST / Spring Web** : permet d'exposer des services accessibles par HTTP (GET, POST, PUT, DELETE).
- **Tomcat embarqué** : serveur web intégré, donc pas besoin d'en installer un.
- **Jackson** : conversion automatique entre objets Java et JSON.
- **DevTools** : redémarrage automatique de l'application pendant le développement.
- **application.properties** : fichier de configuration de l'application (port, etc.).
- **JAR exécutable** : l'application packagée dans un seul fichier, lançable avec `java -jar`.

Résultat attendu

L'application démarre sur `http://localhost:8080`. Comme aucun contrôleur n'existe encore, le navigateur affiche la page **Whitelabel Error Page** : c'est normal, cela prouve que le serveur fonctionne.

B. Démarche pas à pas

I. Télécharger et lancer Spring Tools 4 (STS)

1. Aller sur https://spring.io/tools et télécharger **Spring Tools 4 for Eclipse**, version correspondant à votre système d'exploitation. Vous obtenez une archive.

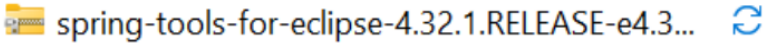


Figure : archive téléchargée (spring-tools-for-eclipse-4.32.1.RELEASE)

2. Dézipper l'archive : un dossier `sts-4.32.1.RELEASE` est créé.



Figure : dossier obtenu après décompression

3. Ouvrir ce dossier : on y trouve configuration, plugins, features, SpringToolSuite4 (exécutable), etc.

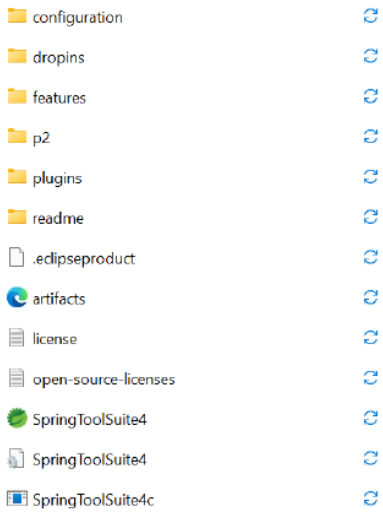


Figure : contenu du dossier sts-4.32.1.RELEASE

4. Double-cliquer sur **SpringToolSuite4.exe** (STS.exe). Choisir un dossier de travail (workspace) si demandé. Après quelques secondes, l'interface Eclipse personnalisée apparaît avec le *dashboard* Spring.

II. Créer le projet Spring Boot

5. Dans STS : **File** → **New** → **Spring Starter Project**.

6. Remplir les champs du formulaire :

Champ	Valeur
Name	Microcommerce
Type	Maven
Packaging	Jar
Java Version	17 ou 21 (selon votre JDK)
Language	Java
Group	com.ecommerce
Artifact	Microcommerce
Package	com.ecommerce.microcommerce

7. Cliquer sur **Next** puis cocher les dépendances : **Spring Web** (pour créer l'API REST) et **Spring Boot DevTools** (rechargement automatique). Elles sont ajoutées au `pom.xml`. Cliquer sur **Finish**.

8. Observer la structure générée dans le *Package Explorer* :

- `pom.xml` : configuration Maven ;
- le package `com.ecommerce.microcommerce` ;
- la classe `MicrocommerceApplication.java`, annotée `@SpringBootApplication` et contenant la méthode `main()`, point d'entrée de l'application.

9. Déplier **Maven Dependencies** pour repérer : Jackson (JSON), Tomcat embarqué (serveur), Logback (logs). Hibernate n'apparaît pas encore : il viendrait avec Spring Data JPA.

III. Exécuter et tester l'application

10. Clic droit sur `MicrocommerceApplication.java` → **Run As** → **Java Application**.

11. Vérifier dans la console le message : `Tomcat started on port(s): 8080 (http)`. L'application tourne sur `http://localhost:8080`.

12. Ouvrir un navigateur à l'adresse `http://localhost:8080`. Résultat attendu : **Whitelabel Error Page**.

Pourquoi cette erreur ? Spring Boot reçoit la requête HTTP mais aucune classe `@Controller` ou `@RestController` ne la traite : il n'y a encore ni endpoint REST (ex. `@GetMapping("/")`) ni page HTML. La page d'erreur est générée automatiquement par Spring Boot.

IV. Générer et exécuter le JAR

13. Clic droit sur le projet → **Run As** → **Maven install**. Maven compile le code et crée le fichier `microcommerce-0.0.1-SNAPSHOT.jar` dans le dossier `target/`.

14. Arrêter l'application lancée dans STS (sinon le port 8080 est occupé), ouvrir un terminal dans le dossier du projet et exécuter :

```
java -jar target/microcommerce-0.0.1-SNAPSHOT.jar
```

L'application démarre comme avant, avec Tomcat sur le port 8080.

Pourquoi utiliser un JAR ? (but)

Un **JAR** (Java ARchive) est un fichier unique qui regroupe le code compilé de l'application, ses ressources et toutes ses dépendances (Spring, Jackson, Tomcat embarqué...). Avec Spring Boot, on obtient un **JAR exécutable**. Le but est de passer du mode développement au mode production :

- **Fonctionner sans STS** : l'application se lance avec une simple commande, sans IDE.
- **Livrer un seul fichier** : facile à copier, à envoyer ou à archiver.
- **Aucun serveur à installer** : Tomcat est déjà inclus dans le JAR.
- **Portabilité** : il tourne sur toute machine (Windows, Linux, serveur, cloud) où Java est installé.
- **Déploiement simplifié** : c'est le format utilisé pour mettre l'application sur un serveur ou dans un conteneur (Docker).
- **Même résultat qu'en développement** : on vérifie que l'application démarre de la même façon (Tomcat sur le port 8080), ce qui prouve qu'elle est prête à être livrée.

V. Personnaliser la configuration

15. Ouvrir `src/main/resources/application.properties` et ajouter :

```
server.port=9090
```

16. Enregistrer et redémarrer l'application : elle est maintenant accessible sur **http://localhost:9090**.

C. Résumé et suite

- Installer l'IDE adapté à Spring Boot (STS).
- Créer un projet via Spring Initializr (intégré à STS).

- Comprendre la structure d'une application Spring Boot.
- Exécuter l'application en développement et en production (JAR).
- Configurer l'application avec `application.properties`.

Prochaines étapes (pour le CRUD) : créer la classe `Produit` (entité), créer un `ProduitController` avec `@RestController`, puis implémenter les méthodes GET, POST, PUT et DELETE.